



Beyond IDOR: The Guide to Advanced Broken Access Control

Stop looking for `id=1`. Start looking for the logic flaws that scanners miss.

Abhishek meena

Published December 3, 2025

Updated December 5, 2025

Free: No

[Read on Freedium](#) · [original](#)

Contents

1. The "Leaky ID" Strategy	4
Where to Hunt for Leaked GUIDs:	4
2. Cross-Tenant Data Leakage (The SaaS Killer)	5
The "Global Object" Mistake	5
3. Blind IDORs & Side Channels	5
Timing Attacks	5
State Changes	6
4. Advanced Bypass Wrappers	6
Parameter Pollution	6
JSON Type Juggling	6
Summary: The Mindset Shift	6

If you've been hunting on bug bounty platforms for more than a month, you've likely noticed a trend: the easy IDORs are gone.

Five years ago, changing `user_id=100` to `user_id=101` in the URL was a valid strategy. Today, frameworks like Rails, Django, and modern SaaS architectures have made basic object reference vulnerabilities rarer. Most applications now use GUIDs (Globally Unique Identifiers) like `550e8400-e29b-41d4-a716-446655440000` specifically to prevent you from guessing other users' IDs.

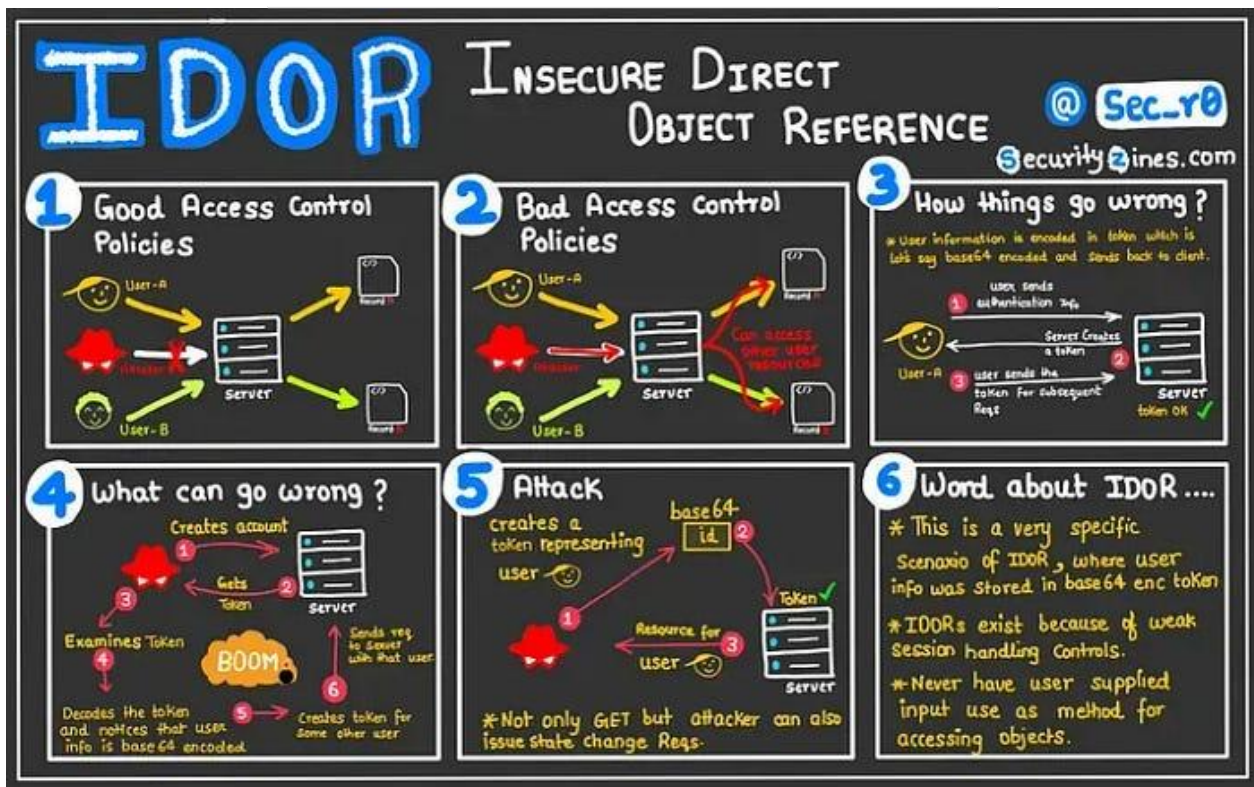
FREE READ : [HERE](https://freedium-mirror.cfd/https://medium.com/@Aacle/beyond-idor-the-guide-to-advanced-broken-access-control-85b7a201baa7?sk=345966b646f7577b083bd0f6235ae79e) (https://freedium-mirror.cfd/https://medium.com/@Aacle/beyond-idor-the-guide-to-advanced-broken-access-control-85b7a201baa7?sk=345966b646f7577b083bd0f6235ae79e)

But here is the secret: **Unpredictable IDs are not Access Control.**

Security through obscurity (using a long, random string) is not a security control. If you can find a way to leak those GUIDs, the application is often just as vulnerable as if it were using integers.

According to the [Bugcrowd Vulnerability Rating Taxonomy \(VRT\)](https://bugcrowd.com/vulnerability-rating-taxonomy) (https://bugcrowd.com/vulnerability-rating-taxonomy), these aren't just generic bugs. They fall under high-priority categories like [Cross-Tenant PII Leakage/Exposure](https://bugcrowd.com/vulnerability-rating-taxonomy) (https://bugcrowd.com/vulnerability-rating-taxonomy) and [Modify/View Sensitive Information \(Complex Object Identifiers\)](https://bugcrowd.com/vulnerability-rating-taxonomy) (https://bugcrowd.com/vulnerability-rating-taxonomy). These are often critical findings because they demonstrate a fundamental failure in the application's authorization logic.

Here is your blueprint for hunting advanced Broken Access Control (BAC) vulnerabilities in modern applications.



1. The "Leaky ID" Strategy

The Myth: "I can't brute force a GUID, so this endpoint is secure." **The Reality:** The app likely leaks GUIDs in places developers forgot to check.

You don't need to guess a 128-bit number. You just need the application to tell you what it is. Developers often scrub IDs from the visible UI but forget about the "invisible" data layers.

Where to Hunt for Leaked GUIDs:

- **Mobile API Traffic:** Mobile apps often fetch massive JSON objects and only display 10% of the data. Proxy the mobile traffic and look for "cousin" objects — users in the same group, organization, or team.
- **Export Features:** This is a goldmine. If an app allows you to "Export Team to CSV/PDF," trigger it. The metadata in that file often contains the GUIDs of every user in the org.
- **Error Messages:** Try to register an account with an email that already exists. A poor implementation might return:

```
{  "error": "User already exists",  "existing_user_id": "771e8400-e29b..." }
```

□ **Toolkit:** You don't have to search manually. You can automate this using [Burp Suite BChecks](https://github.com/PortSwigger/BChecks) (<https://github.com/PortSwigger/BChecks>) to write custom scripts that highlight regex patterns matching UUIDs automatically in HTTP responses.

2. Cross-Tenant Data Leakage (The SaaS Killer)

In B2B applications (Slack, Jira, Salesforce-style apps), the "Tenant" is the holy grail. A vulnerability that allows Tenant A to see Tenant B's data is almost always a P1 or P2.

This concept maps directly to OWASP's [API1:2023 Broken Object Level Authorization \(BOLA\)](https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/) (<https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>). The VRT specifically prioritizes "**Cross-Tenant PII Leakage/Exposure**" as a Priority 1 issue because the business impact is devastating.

The "Global Object" Mistake

Developers often create objects that are intended to be shared globally (like "System Tags" or "Default Categories") but accidentally mix them with private customer data.

- **Scenario:** You are editing a "Label" in your dashboard. The request looks like:
`GET /api/v1/labels/custom_label_id`
- **The Attack:** Try to fuzz that ID. If the backend code looks like: `SELECT * FROM labels WHERE id = input_id ...instead of... SELECT * FROM labels WHERE id = input_id AND tenant_id = current_user.tenant_id` ...you might pull up internal labels from another company.

3. Blind IDORs & Side Channels

Sometimes, the door is open, but the room is dark. You change the ID to a victim's ID, and the server responds with `200 OK`, but the response body is empty. Is it vulnerable?

You need to look for **Side Channels**.

Timing Attacks

Does the database take longer to process a valid ID than an invalid one?

- `GET /users/VALID_ID -> 200ms`
- `GET /users/INVALID_ID -> 50ms`

This discrepancy suggests the server is performing operations (fetching data, processing logic) before it realizes you aren't authorized to see it.

State Changes

If you send a `DELETE /users/VICTIM_ID` and get a `200 OK` (even with no body), log into the victim account. Is the user deleted? Is the account locked?

□ **Practice Lab:** You can try this exact scenario in PortSwigger's free [Insecure Direct Object References \(IDOR\) Lab](https://portswigger.net/web-security/access-control/idor) (<https://portswigger.net/web-security/access-control/idor>) to see how invisible changes can still be tracked.

4. Advanced Bypass Wrappers

When the front door is locked, try the window. WAFs and basic security filters often look for specific patterns. You can confuse them with "Parameter Pollution" or Type Juggling.

Parameter Pollution

Most frameworks handle duplicate parameters differently.

- **Request:** `GET /api/messages?user_id=VICTIM&user_id=ATTACKER`
- **Logic:** The security check might validate the second ID (Attacker = Authorized), but the business logic might fetch data for the first ID (Victim).
- **Deep Dive:** To understand which servers (PHP vs Python vs Node) are vulnerable to this, check out the [OWASP HTTP Parameter Pollution Guide](https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/04-Testing_for_HTTP_Parameter_Pollution) (https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/04-Testing_for_HTTP_Parameter_Pollution).

JSON Type Juggling

If an endpoint expects a number, send an array or an object.

- **Original:** `{"id": 123}`
- **Bypass:** `{"id": [123]}`
- **Why it works:** In Javascript/Node.js, `[123] == 123` is often true, but strict access control libraries might fail to match the array format against the deny list. For more on this, read about [Node.js Input Mistakes](https://freedium-mirror.cfd/https://medium.com/@Krutik090/the-5-node-js-input-mistakes-attackers-love-and-how-to-fix-them-cc42474e8a1d) (<https://freedium-mirror.cfd/https://medium.com/@Krutik090/the-5-node-js-input-mistakes-attackers-love-and-how-to-fix-them-cc42474e8a1d>).

Summary: The Mindset Shift

The days of simple ID swapping are ending. But the era of **Logic Hacking** is just beginning.

To find these bugs, you have to stop thinking like a user and start thinking like a developer who is tired, rushing to meet a deadline, and copying code from StackOverflow. They assume that if you don't know the ID, you can't access the data.

Your job is to prove them wrong.